Applying Discrete Modeling Techniques

to Real-World Computing Problems

Zachery Thompson

CS 505: Discrete Structures

Southern New Hampshire University

Dr. Jeffrey Zheng

May 29, 2026

Applying Discrete Modeling Techniques to Real-World Computing Problems

Introduction

Discrete modeling techniques are widely used in computer science to analyze systems involving probability, relationships, sequencing, and changing states. These models help developers and analysts better understand complex problems that may be difficult to evaluate through simple calculations alone. From entertainment systems to education and workplace management, discrete modeling techniques provide structured approaches for analyzing data, predicting outcomes, and improving decision-making. The following sections examine three real-world computing problems and explore how discrete event simulation, Markov chains, and graph theory can be applied to address them.

Daily Life

Computing Problem

One real-world computing problem involves predicting and balancing combat encounters in tabletop role-playing games such as Dungeons & Dragons and Daggerheart. Game masters must manage combat difficulty, resource usage, encounter pacing, and player progression while accounting for many changing variables. Character abilities, enemy statistics, dice randomness, initiative order, and player decision-making all influence combat outcomes simultaneously. As encounters become more complex, predicting outcomes through simple calculations alone becomes increasingly unreliable.

Modeling Technique: Discrete Event Simulation

Discrete-event simulation is well suited to this problem because tabletop combat unfolds as a sequence of individual events over time. According to Vito Stamatti (2024), discrete-event simulation models dynamic systems by tracking individual events that occur at distinct points in time. In tabletop role-playing games, these events include initiative rolls, attacks, healing, spell usage, and character defeat. Since each action changes the state of combat and influences future outcomes, the encounter can be modeled as a system of sequential probabilistic events. Stamatti (2024) further explains that discrete-event simulation is particularly useful for systems involving entities, events, queues, and changing states over time, which aligns closely with turn-based tabletop combat systems.

Application/Solution

By repeatedly simulating combat encounters, a game master can estimate encounter difficulty, combat duration, and player survival rates before running the session. This helps address the challenge of balancing encounters involving many interconnected variables and unpredictable outcomes. Discrete-event simulation is especially valuable for modeling complex dynamic systems because it enables analysts to evaluate system behavior under different conditions and scenarios (Stamatti, 2024). One advantage of discrete-event simulation is that it can model randomness and changing game states more realistically than static formulas alone. Additionally, simulations allow experimentation without affecting actual gameplay, making them useful for testing encounter balance before implementation. However, the accuracy of the simulation depends heavily on the assumptions and rules built into the model, and large numbers of simulations may be required to produce reliable results. Stamatti (2024) also notes that discrete-event simulation models may become computationally intensive and increasingly complex as systems grow larger and contain more interacting variables.

Rationale

Discrete-event simulation aligns closely with the structure of tabletop combat because encounters naturally progress through turn-based actions and state changes over time. Stamatti (2024) explains that discrete-event simulation is designed to model systems where events occur sequentially and directly influence future system states. This makes the technique particularly effective for tabletop combat systems, where initiative order, player decisions, resource usage, and dice rolls all affect subsequent actions and outcomes. The technique also addresses the randomness introduced by player choices and probabilistic mechanics, enabling efficient analysis of many possible encounter outcomes before gameplay begins.

Education or Studies

Computing Problem

Another computing problem involves modeling educational progression within a Master of Science in Computer Science program. Managing coursework, assignment scheduling, studying, and long-term skill development requires balancing multiple priorities and deadlines. Additionally, learning does not occur in a perfectly linear manner. Students often revisit earlier material before mastering advanced concepts, and educational progression frequently involves shifts in understanding over time.

Modeling Technique: Markov Chains

Markov chains are appropriate for this problem because they model systems that transition between states according to probability. Halder (2025) explains that a Markov chain describes a system that moves between states, in which the probability of the next state depends on the current state. In education, these states may represent levels of understanding, such as limited, partial, proficient, and mastered. The probability of transitioning between these states can be analyzed to model student progression over time.

Application/Solution

Markov chains can help model how students progress through material and identify where additional reinforcement or intervention may be needed. This addresses the challenge of predicting educational outcomes in systems where learning is gradual and non-linear. One advantage of Markov chains is that they simplify probabilistic systems into measurable state transitions that can be analyzed mathematically. However, human learning is influenced by many factors, such as stress, motivation, external responsibilities, and individual learning differences; a simplified probabilistic model may not fully capture that. This limitation aligns with Halder's (2025) discussion that Markov chains can be limited when transition probabilities are fixed or when systems involve more complex dependencies.

Rationale

Markov chains are appropriate because educational progression can be represented as movement between different states of understanding. Since future academic progress is often influenced by a student's current level of understanding, the structure of Markov chains aligns naturally with the progression of learning. The technique also helps simplify complex educational behaviors into manageable and measurable patterns. As Halder (2025) notes, Markov chains are useful because of their simplicity and ability to model future probabilities through state transitions.

Career

Computing Problem

A third computing problem involves managing inventory systems within a middle school instrumental music program. Instruments, accessories, repair records, and student assignments must frequently be tracked across multiple grade levels and ensembles. Managing this information manually can quickly become inefficient and error-prone, especially when equipment regularly moves between students, classrooms, storage areas, performances, and repair shops.

Modeling Technique: Graph Theory

Graph theory is appropriate for this problem because inventory systems rely heavily on relationships between connected entities. According to Vaidehi Joshi (2017), graphs are used to represent networks composed of interconnected objects formally. In this inventory system, students, instruments, classrooms, and repair locations can be represented as vertices, while assignments and movements between them can be represented as edges. This creates a structured network of interconnected relationships that can be analyzed and tracked efficiently.

Application/Solution

Graph theory allows the inventory system to efficiently track equipment assignments, locations, and movement throughout the school year. This helps solve the challenge of organizing and analyzing large amounts of interconnected information. One advantage of graph theory is that it provides a clear representation of relationships between entities, improving searching and relationship tracking. Joshi (2017) explains that graphs are particularly useful for representing systems where many objects interact through connected relationships, such as social networks and web structures. Another advantage is that graph-based systems scale effectively as additional students and instruments are added. However, graph structures may become increasingly complex as systems grow larger, requiring careful organization and validation to maintain accuracy.

Rationale

Graph theory aligns naturally with inventory management because the system is fundamentally based on relationships between interconnected entities. Since graph theory is specifically designed to model networks and connected systems, it fits well with tracking how instruments move between students, classrooms, storage areas, and repair locations. The technique also addresses scalability constraints by providing an efficient method to organize and analyze increasingly complex relationships as the inventory system expands. As Joshi (2017) notes, graphs are especially valuable for representing systems in which connections among entities are more important than strict hierarchical organization.

Conclusion

Discrete modeling techniques provide valuable tools for analyzing and solving real-world computing problems across many different environments. Discrete event simulation supports the analysis of sequential and probabilistic systems such as tabletop combat encounters, Markov chains help model changing states within educational progression, and graph theory provides structured methods for managing interconnected inventory systems. Although each technique contains limitations and assumptions, these models help simplify complex problems into organized systems that can be analyzed more effectively. As computer science continues to expand into increasingly data-driven and interconnected environments, discrete modeling techniques will remain important tools for problem-solving and decision-making.

References

Halder, S. (2025, July 31). Markov chain. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/markov-chain/

Joshi, V. (2017, March 20). A gentle introduction to graph theory. Medium. https://medium.com/basecs/a-gentle-introduction-to-graph-theory-77969829ead8

Stamatti, V. (2024, January 12). Introduction to discrete event simulation with Python. Medium. https://medium.com/@vitostamatti1995/introduction-to-discrete-event-simulation-with-python-3b0cce67f92e